



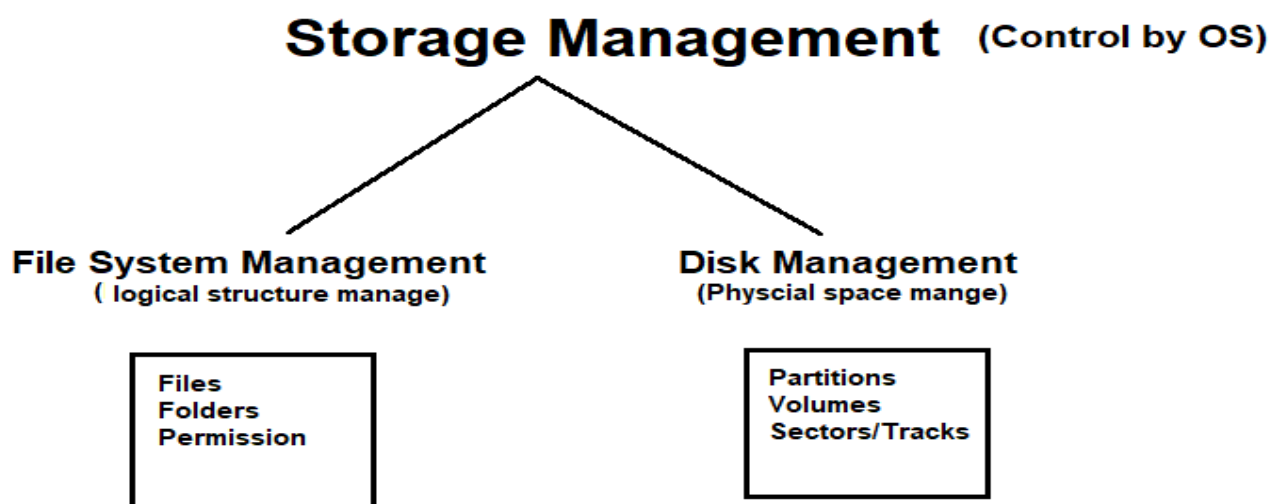
# S.P.M College, Udantpuri

Bachelor Of Computer Application (BCA)

Part -1 (Paper-2)

– Hira Kumar

- **Data** → Data is any collection of raw, unorganized facts, figures, symbols, or signals that a computer can process, store and transmit.  
**Example :-** alphabets (A-Z, a-z), digits (0-9) and using special characters (+, -, ., #, \$, etc)
- **Instructions** → Step by step commands telling a computer what to do.  
**Example :-** Add 25+27, Store/save file.
- **Information** → Processed data that is meaningful and useful (i.e. final result), is called information.  
**Example :-** “The age of Ram is 25”.
- **File** → A file is a named collection of related information that is recorded on secondary storage such as magnetic disks, magnetic tapes and optical disks.



## ❖ File Management

- File management is the systematic process of organizing, storing, retrieving, naming, securing and manipulating files and folders on a computer's storage system to ensure efficient access, data integrity, and optimal storage utilization.
- A file is a named collection of related information that is recorded on secondary storage such as magnetic disks, magnetic tapes and optical disks.
- In general, a file is a sequence of bits, bytes, lines or records whose meaning is defined by the files creator and user.

## ❖ Attributes / Properties of a File

File related data (Data about data is called meta data) means File's name, extension, location, size, protection, encryption,...)

Following are some of the attributes of a file:-

- **Name** = It is the only information which is in human-readable form.
- **Identifier** = The file is identified by a unique tag(number) within file system.
- **Type** = It is needed for systems that support different types of files.
- **Location** = Pointer to file location on device.
- **Size** = The current size of the file.
- **Protection** = This controls and assigns the power of reading, writing, executing.

- **Time, date, and user identification** This is the data for protection, security, and usage monitoring.

## ❖ File Operations

- **Creating a file:** Two steps are necessary to create a file. First, space in the file system must be found for the file. Second, an entry for the new file must be made in the directory.
- **Writing a file:** To write a file, we make a system call specifying both the name of the file and the information to be written to the file. Given the name of the file, the system searches the directory to find the file's location. The system must keep a write pointer to the location in the file where the next write is to take place. The write pointer must be updated whenever a write occurs.
- **Reading a file:** To read from a file, we use a system call that specifies the name of the file and where (in memory) the next block of the file should be put. Again, the directory is searched for the associated entry, and the system needs to keep a read pointer to the location in the file where the next read is to take place. Once the read has taken place, the read pointer is updated.
- **Repositioning within a file:** The directory is searched for the appropriate entry, and the current-file-position pointer is repositioned to a given value. Repositioning within a file need not involve any actual I/O. This file operation is also known as a file seek.
- **Deleting a file:** To delete a file, we search the directory for the named file. Having found the associated directory entry, we release all file space, so that it can be reused by other files, and erase the directory entry.
- **Protection:** Access-control information determines who can do reading, writing, executing, and so on.

- **Truncating a file:** The user may want to erase the contents of a file but keep its attributes. Rather than forcing the user to delete the file and then recreate it, this function allows all attributes to remain unchanged—except for file length—but lets the file be reset to length zero and its file space released.

## Types of most popular file extensions

We will discuss the popular types of file extensions listed below in this article.

**Text and word processing files:** .doc, .docx, .odt, .txt, .pages

**Web-related files:** .htm, .html

**Page layout files:** .pdf

**Spreadsheet files:** .xls, .xlsx, .ods

**Presentation files:** .ppt, .pptx, .key

**Image files:** .bmp, .jpeg, .jpg, .png

**Programming files:** .py, .java, .js

**Compression and archive files:** .zip

**Executable program files:** .exe, .app, .bin

**Audio and video files:** .mp3, .wav, .mp4

**Draw program files:** .drw, .dwg

**System files:** .sys, .tmp, .ini

## ❖ File Access method

- File access mechanism refers to the manner in which the records of a file may be accessed.
- There are several ways to access files
  - ✓ **Sequential access**
  - ✓ **Direct/Random access**
  - ✓ **Indexed sequential access**

## 1. Sequential Access

- A sequential access is that in which the records are accessed in some sequence, i.e., the information in the file is processed in order, one record after the other.
- This access method is the most primitive one. The idea of Sequential access is based on the tape model which is a sequential access device.
- The Sequential access method is best because most of the records in a file are to be processed. For example, transaction files.

### Advantages of sequential access

- ✓ It is simple to program and easy to design.
- ✓ Sequential file is best use if storage space.

### Disadvantages of sequential access

- ✓ Sequential file is time consuming process.
- ✓ It has high data redundancy.
- ✓ Random searching is not possible.

## 2. Direct Access/Random Access

- Sometimes it is not necessary to process every record in a file.
- It is not necessary to process all the records in the order in which they are present in the memory. In all such cases, direct access is used.
- The disk is a direct access device which gives us the reliability to random access of any file block.
- In the file, there is a collection of physical blocks and the records of that blocks.
- Example: Databases are often of this type since they allow query processing that involves immediate access to large amounts of information. All reservation systems fall into this category.

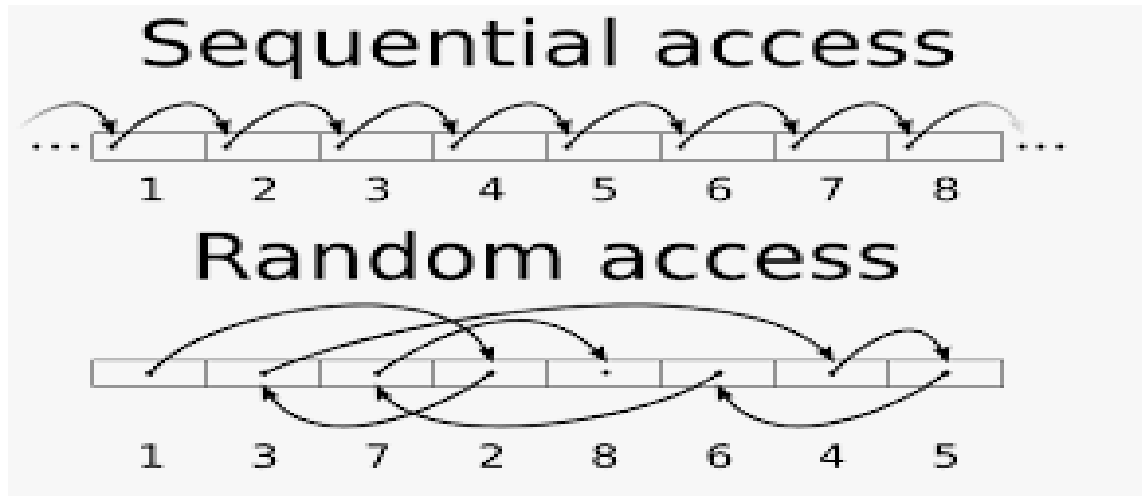
### Advantages:

- ✓ Direct access file helps in online transaction processing system (OLTP) like online railway reservation system.
- ✓ In direct access file, sorting of the records are not required.
- ✓ It accesses the desired records immediately.

- ✓ It updates several files quickly.
- ✓ It has better control over record allocation.

## Disadvantages:

- ✓ Direct access file does not provide backup facility.
- ✓ It is expensive.
- ✓ It has less storage space as compared to sequential file



## 3. Index sequential access

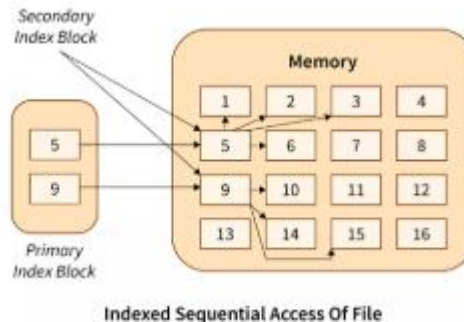
- The index sequential access method is a modification of the direct access method. Basically, it is kind of combination of both the sequential access as well as direct access.
- The main idea of this method is to first access the file directly and then it accesses sequentially.
- The direct access of the index is made to access a record in a file.
- The information which is obtained from this access is used to access the file. Sometimes the indexes are very big.
- It uses an Index to control the pointer while accessing files.

## Advantages:

- ✓ In indexed sequential access file, sequential file and random file access is possible.
- ✓ It accesses the records very fast if the index table is properly organized.
- ✓ The records can be inserted in the middle of the file.
- ✓ It provides quick access for sequential and direct processing.
- ✓ It reduces the degree of the sequential search.

## Disadvantages:

- ✓ Indexed sequential access file requires unique keys and periodic reorganization.
- ✓ Indexed sequential access file takes longer time to search the index for the data access or retrieval.
- ✓ It requires more storage space.
- ✓ It is expensive because it requires special software.



## ❖ File Allocation Methods (FAM) / File Space Allocation

- An allocation method refers to how disk blocks are allocated for files.

### FAM have two benefits –

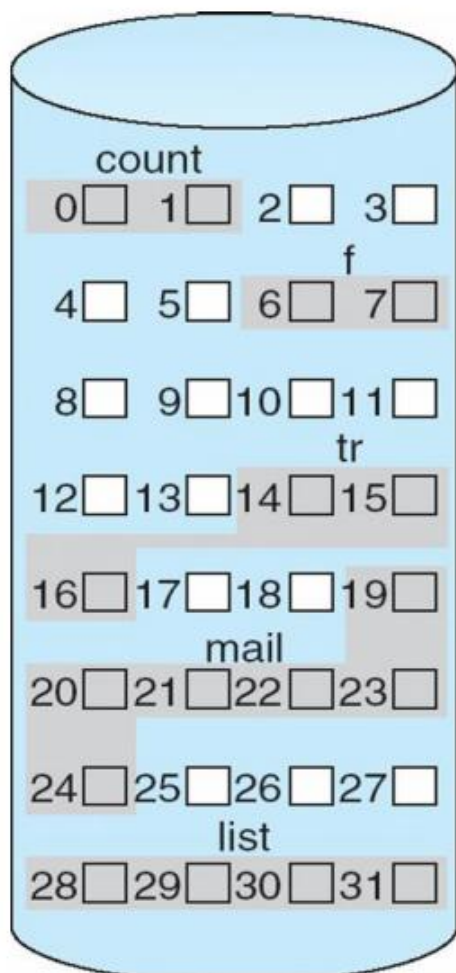
- i. Efficiency Disk Utilization
- ii. File/Data Access Faster

### ➤ Three Types of FAM

1. Contiguous allocation
2. Linked allocation
3. Indexed allocation

## 1. Contiguous allocation

Each file occupies a set/Sequence of contiguous blocks on the disk.



| directory |       |        |
|-----------|-------|--------|
| file      | start | length |
| count     | 0     | 2      |
| tr        | 14    | 3      |
| mail      | 19    | 6      |
| list      | 28    | 4      |
| f         | 6     | 2      |

The directory entry for a file with contiguous allocation contains

1. Address of starting block
2. Length of the allocated portion.

This Figure → The file 'mail' in the following figure starts from the block 19 with length = 6 blocks. Therefore, it occupies 19, 20, 21, 22, 23, 24 blocks.

## Advantages:

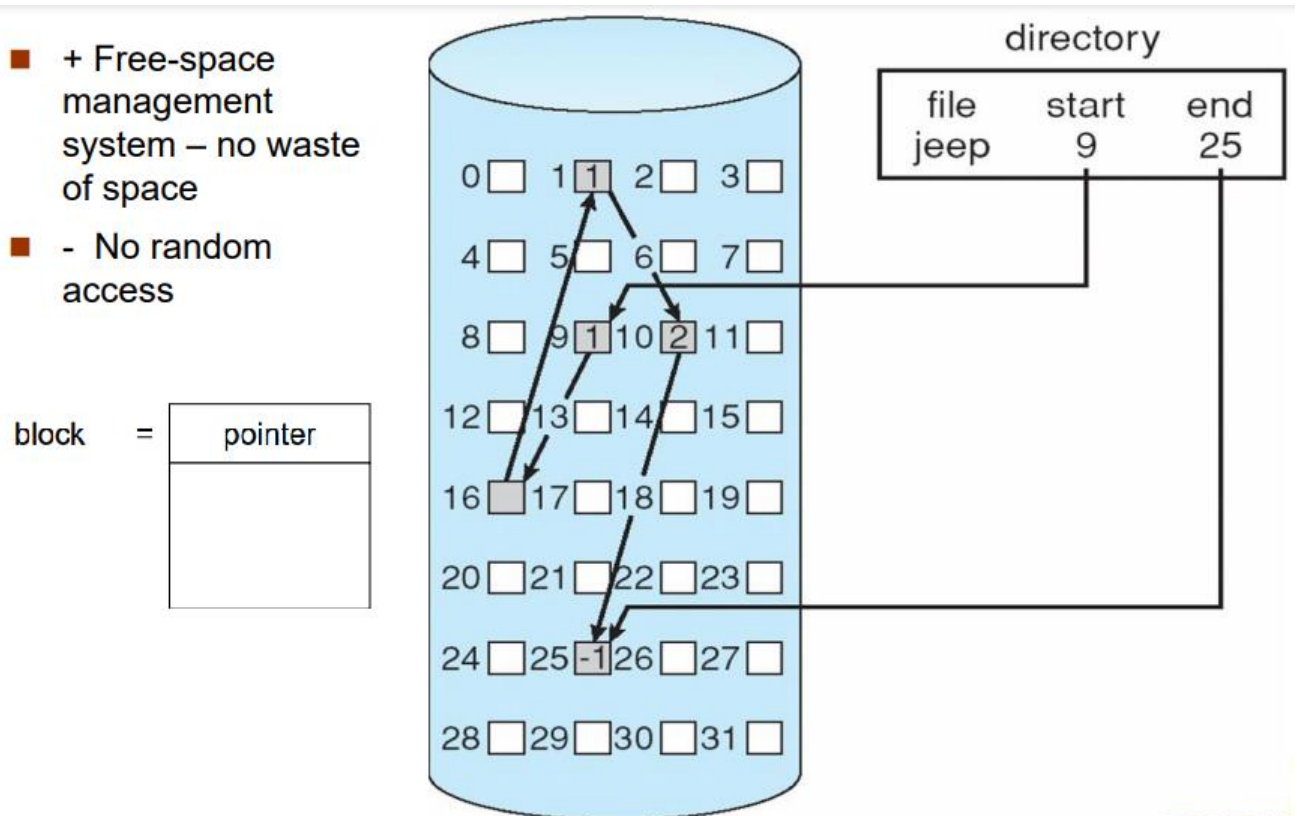
- ✓ Both the Sequential and Direct Accesses are supported by this.
- ✓ For direct access, the address of the kth block of the file which starts at block b can easily be obtained as  $(b+k)$ .
- ✓ This is extremely fast since the number of seeks are minimal because of contiguous allocation of file blocks.

## Disadvantages:

- ✓ This method suffers from both internal and external fragmentation. This makes it inefficient in terms of memory utilization.
- ✓ Increasing file size is difficult because it depends on the availability of contiguous memory at a particular instance.

## 2. Linked allocation

- In this scheme, each file is a linked list of disk blocks which need not be contiguous.
- The disk blocks can be scattered anywhere on the disk.
- The directory entry contains a pointer to the starting and the ending file block.
- Each block contains a pointer to the next block occupied by the file.



### Advantages:

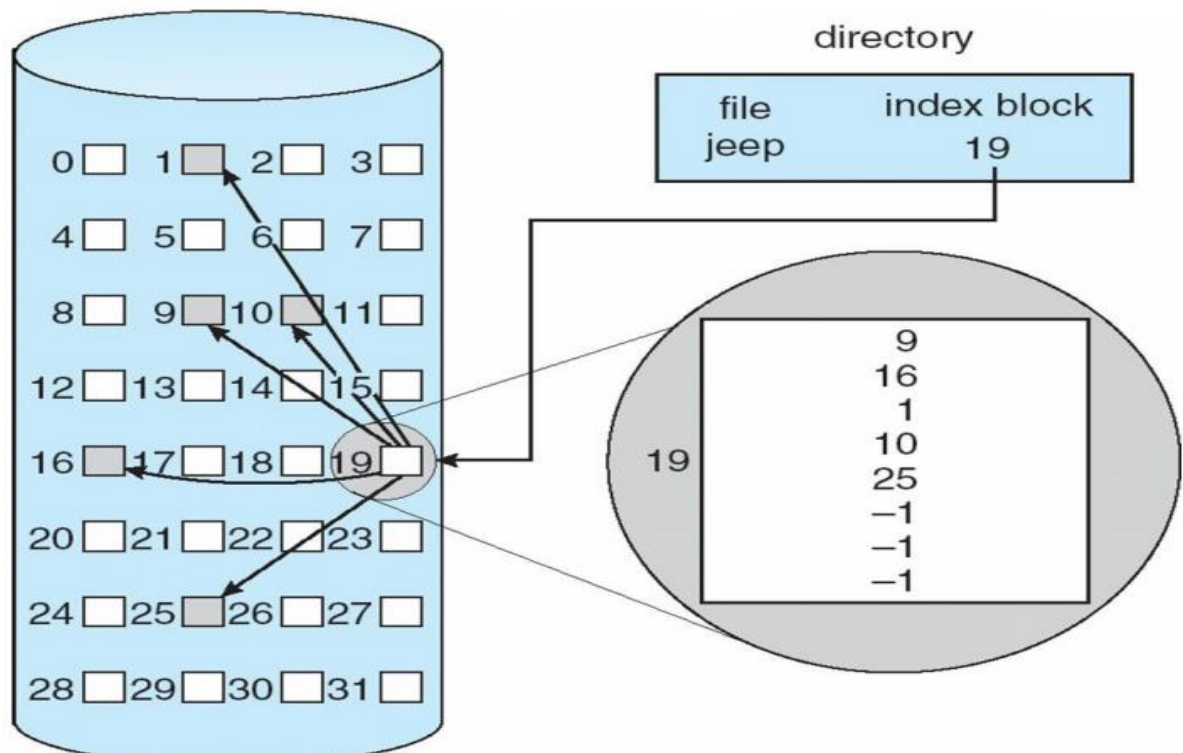
- ✓ File size does not have to be specified.
- ✓ No external fragmentation.

### Disadvantages:

- ✓ It does sequential access efficiently and is not for direct access
- ✓ Each block contains a pointer, wasting space.
- ✓ Blocks scatter everywhere and a large number of disk seeks may be necessary.
- ✓ Reliability: what if a pointer is lost or damaged?

## 3. Indexde Allocation

- In this scheme, a special block known as the Index block contains the pointers to all the blocks occupied by a file<sup>2</sup>. Each file has its own index block.
- The *i*th entry in the index block contains the disk address of the *i*th file block.
- The directory entry contains the address of the index block as shown in the image:
- Each file has its own index block which stores the addresses of disk space occupied by the file. Directory contains the addresses of index blocks of files



## Advantages:

- ✓ This supports direct access to the blocks occupied by the file and therefore provides fast access to the file blocks.
- ✓ It overcomes the problem of external fragmentation.

## Disadvantages:

- ✓ The pointer overhead for indexed allocation is greater than linked allocation.
- ✓ For very small files, say files that expand only 2-3 blocks, the indexed allocation would keep one entire block (index block) for the pointers which is inefficient in terms of memory

utilization. However, in linked allocation we lose the space of only 1 pointer per block.

### ❖ Difference b/w Contiguous, linked & Indexed

| Sr. No. | Contiguous                                       | linked                                                             | Indexed                                       |
|---------|--------------------------------------------------|--------------------------------------------------------------------|-----------------------------------------------|
| 1.      | Allocate each file to contiguous blocks on disk. | Allocate linked-list of fixed-sized blocks.                        | Allocate fixed-sized blocks for each file.    |
| 2.      | It suffers from external fragmentation.          | No external fragmentation.                                         | No external fragmentation.                    |
| 3.      | Pre-allocation is required                       | Pre-allocation is possible.                                        | Pre-allocation is possible.                   |
| 4.      | It support variable size portions.               | It support fixed size portions.                                    | It supports fixed and variable size portions. |
| 5.      | Simple to calculate random addresses.            | Cannot calculate random addresses without reading previous blocks. | Supports random access.                       |
| 6.      | Allocation frequency is once.                    | Allocation frequency is low to high.                               | Allocation frequency is medium.               |
| 7.      | No pointer overhead.                             | Space required for pointer.                                        | Pointer overhead is generally greater.        |

### ❖ Directory /Folder Structure

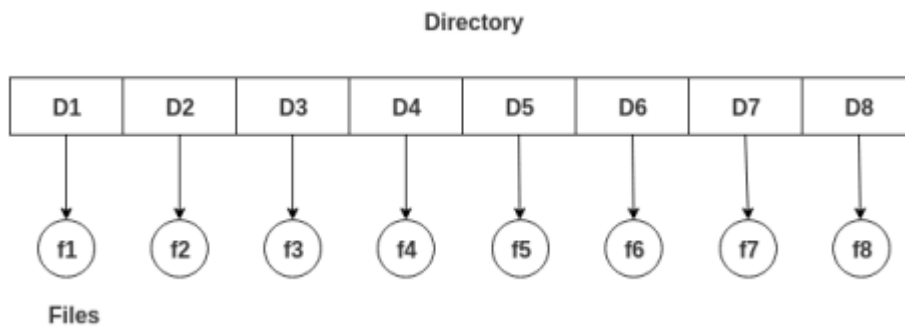
- A directory is a container that is used to contain folders and file.
- It organizes files and folders into a hierarchical manner.

### Three type of Directory structure

1. Single level directory
2. Two level directory
3. Tree level directory

#### 1. Single level directory

- Single level directory is simplest directory structure.
- In it all files are contained in same directory which make it easy to support and understand. Since all the files are in the same directory, they must have the unique name. if two users call their dataset test, then the unique name rule violated.



## Advantages:

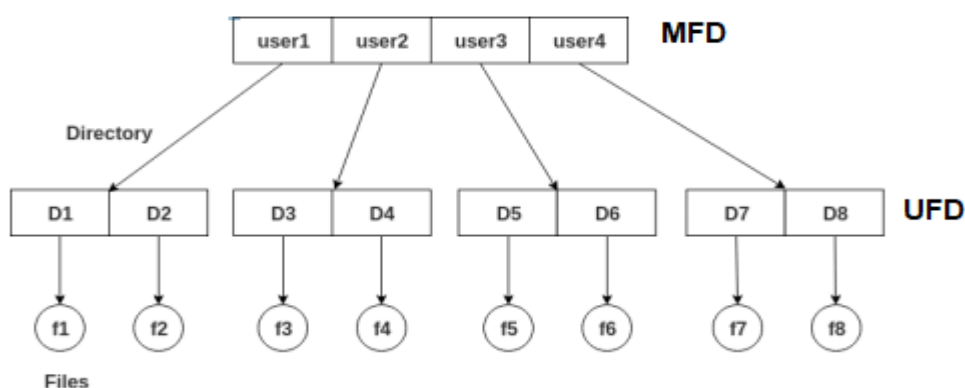
- ✓ Since it is a single directory, so its implementation is very easy.
- ✓ If files are smaller in size, searching will be faster.
- ✓ The operations like file creation, searching, deletion, updating are very easy in such a directory structure.

## Disadvantages:

- ✓ There may be a chance of name collision because two files cannot have the same name.
- ✓ Searching will become time taking if the directory is large.
- ✓ In this, cannot group the same type of files together.

## 2. Two level directory

- As, a single level directory often leads to confusion of file names among different users, hence the solution to this problem is to create a separate directory for each user.
- In the two-level directory structure, each user has their own **user files directory (UFD)**.
- The UFDs have similar structures, but each lists only the files of a single user. The system's **master file directory (MFD)** searches whenever a new user ID is logged in.
- The MFD is indexed by username or account number, and each entry points to the UFD for that user.



Example :-

| MFD contain                                   | Hira UFD1 contain           |
|-----------------------------------------------|-----------------------------|
| Hira → UFD1<br>Abhishek → UFD2<br>Sonu → UFD3 | Bca.txt<br>Bca.c<br>God.mp3 |

## Advantages:

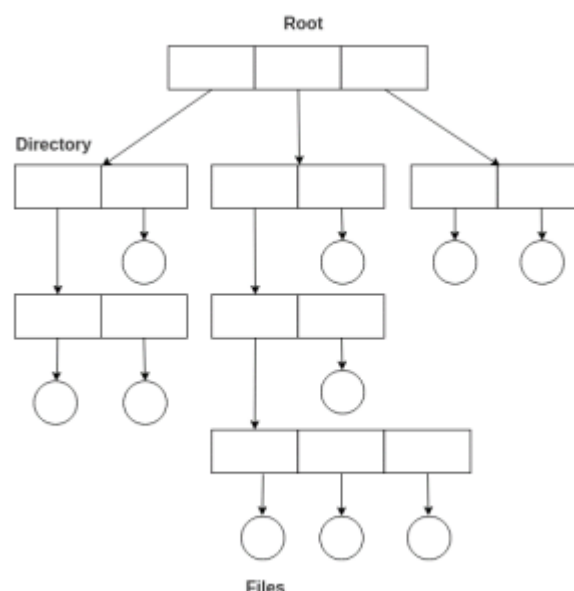
- ✓ We can give full path like /User-name/directory-name.
- ✓ Different users can have same directory as well as file name.
- ✓ Searching of files become more easy due to path name and user-grouping.

## Disadvantages:

- ✓ A user is not allowed to share files with other users.
- ✓ Still it not very scalable, two files of the same type cannot be grouped together in the same user.

## 3. Tree level directory

- Once we have seen a two-level directory as a tree of height 2, the natural generalization is to extend the directory structure to a tree of arbitrary height.
- This generalization allows the user to create their own subdirectories and to organize on their files accordingly.
- A tree structure is the most common directory structure. The tree has a root directory, and every file in the system have a unique path.



## Advantages:

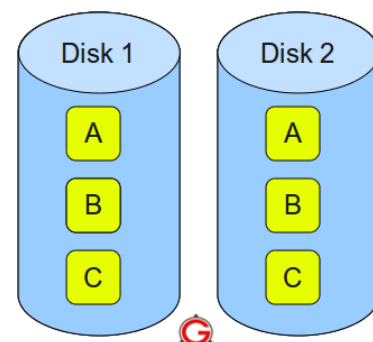
- ✓ Very generalize, since full path name can be given.
- ✓ Very scalable, the probability of name collision is less.
- ✓ Searching becomes very easy, we can use both absolute path as well as relative.

## Disadvantages:

- ✓ Every file does not fit into the hierarchical model; files may be saved into multiple directories.
- ✓ We cannot share files.
- ✓ It is inefficient, because accessing a file may go under multiple directories.

## ❖ RAID in DBMS

- RAID means – Redundant Array of Independent/ Inexpensive Drives
- RAID is multiple disk connected together to increase the performance, data redundancy or both.



**RAID 1** – Blocks Mirrored. No Stripe. No parity.

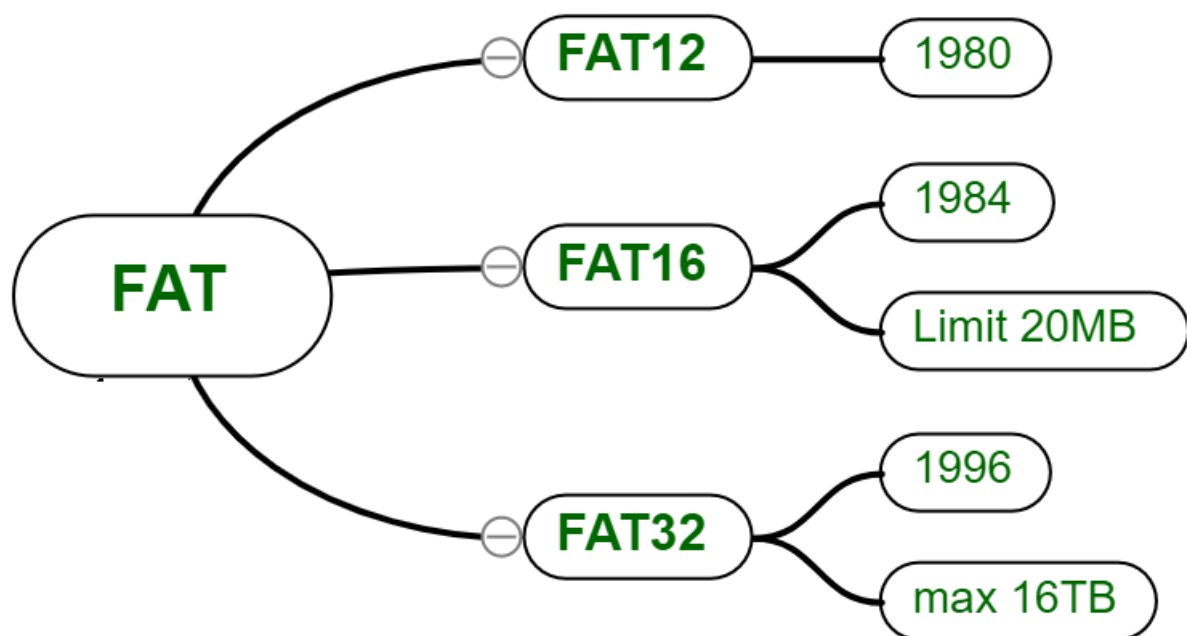
[Note :- Facebook पर, Instagram पर, whatsapp पर, google drive, google photo,..... पर जो photo/video/ फाइल रखते हैं वो कहाँ रखता हैं | उत्तर - colud पर (इंटरनेट पर कुछ रखना या store करना i.e. cloud storage)

Cloud storage मतलब जहाँ इन कंपनी का सर्वर हैं सर्वर मतलब collection of harddisk. तो हम सर्वर ना बोल के colud क्यों बोलते हैं - क्यूकि 1 कंपनी के पास बहुत सारे सर्वर होते हैं और सभी सर्वर पे आपका data save करना i.e. colud storage | भविष्य में 1 सर्वर खराब हो गया तो दुसरे सर्वर से data recover होगा ||

# ❖ OS File System

- A file system is the methods and data structures that an OS uses to keep Sector of files on a disk or partition.
- How to file Store in memory and How to file Fetch in memory i.e. STORE & FETCH
- ✓ **FAT** = File allocation Table (old file system – DOS/Windows-95)
- ✓ **exFAT** = Extensible File Allocation Table
- ✓ **NTFS** = New Technology File System (Use - Windows)
- ✓ **HFS** = Hierarchical File System
- ✓ **EXT** = Extended File System (Linux)

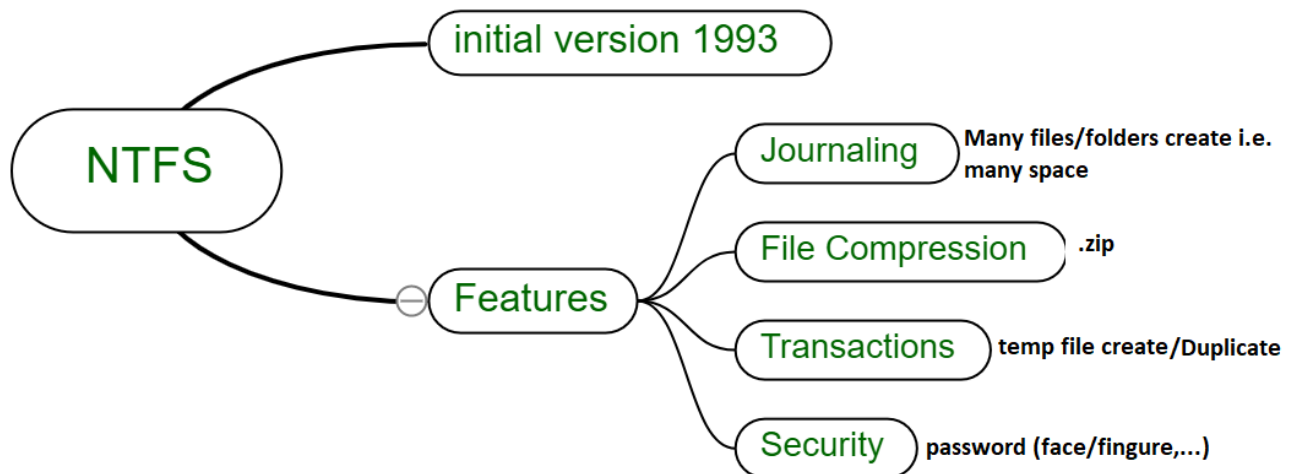
## • FAT



**USE = Windows 98 (fat32), Ms-Dos**

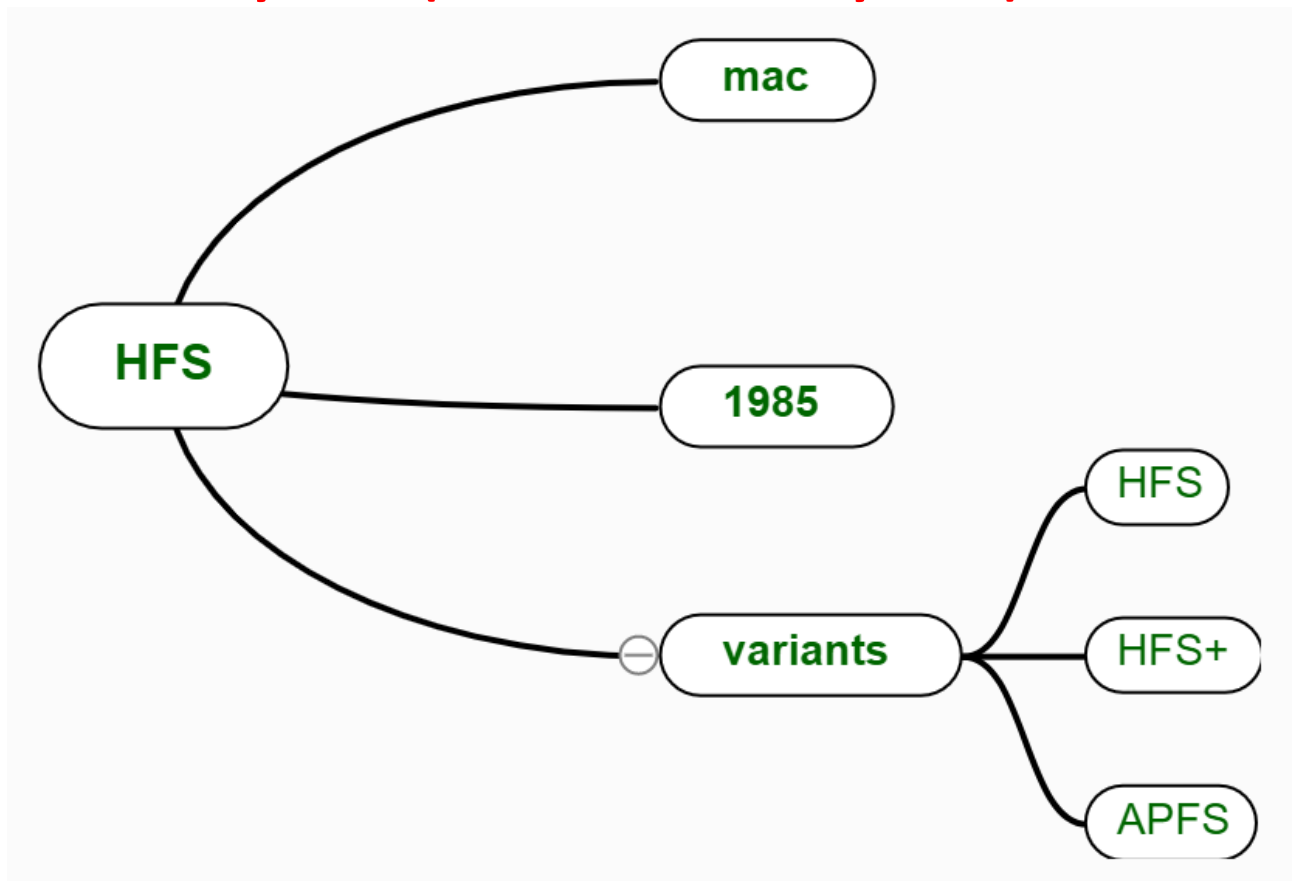
- **ExFAT** = Use in SD Cards & USB Drives(PEN Drive)

## • NTFS File System



e.g – Windows NT/2000/XP/Vista/7/8/10/11

## • HFS File System (Hierarchial file system)

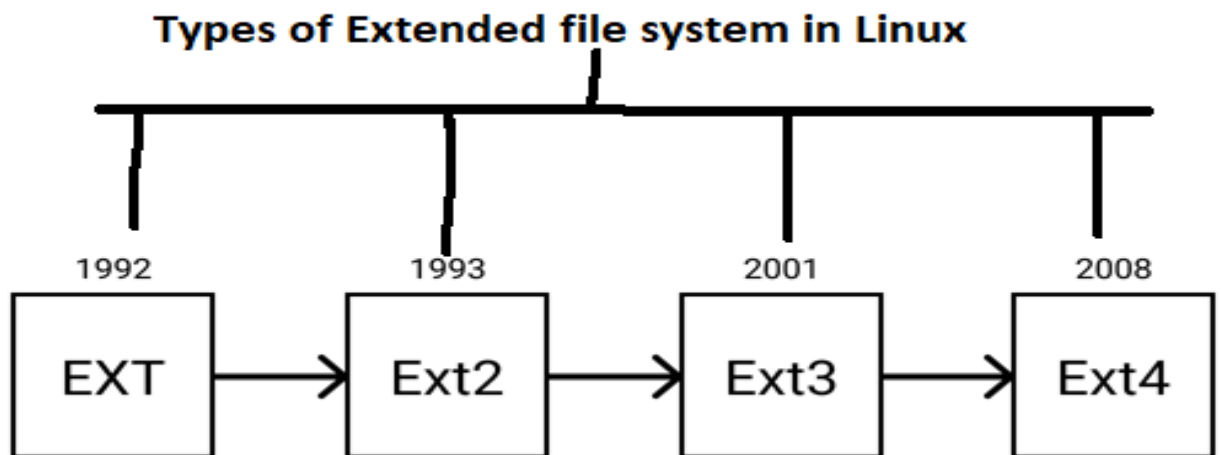


e.g. - Apple Operating System = MAC OS

HFS → mac OS

APFS → ios /mac OS

- **Extended File System**



**e.g - Unix & Linux OS support Extended File System**

Android → FAT32 / Ext3 / Ext4

Linux(Red hat) → XFS (high performance scalable file system)

Linux(Fedora OS) → B-trfs (B-Tree file system)

Oracle -solaris OS → ZFS (zeetabyte file system)

=====HIRA KUMAR=====